

./requirements.txt

./requirements.txt

```
asgiref==3.11.0
async-generator==1.10
attrs==25.4.0
certifi==2026.1.4
cffi==2.0.0
charset-normalizer==3.4.4
Django==6.0.1
h11==0.16.0
idna==3.11
importlib_metadata==8.7.1
mypy_extensions==1.1.0
outcome==1.3.0.post0
packaging==26.0
pycparser==3.0
PySocks==1.7.1
python-dotenv==1.2.1
requests==2.32.5
selenium==4.40.0
sniffio==1.3.1
sortedcontainers==2.4.0
sqlparse==0.5.5
trio==0.32.0
trio-typing==0.10.0
trio-websocket==0.12.2
types-certifi==2021.10.8.3
types-urllib3==1.26.25.14
typing_extensions==4.15.0
tzdata==2025.3
urllib3==2.6.3
webdriver-manager==4.0.2
websocket-client==1.9.0
wsproto==1.3.2
zipp==3.23.0
```

./webscraper_project/manage.py

./webscraper_project/manage.py

```
#!/usr/bin/env python
"""Django's command-line utility for administrative tasks."""
import os
import sys

def main():
    """Run administrative tasks."""
    os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'webscraper_project.settings')
    try:
        from django.core.management import execute_from_command_line
    except ImportError as exc:
        raise ImportError(
            "Couldn't import Django. Are you sure it's installed and "
            "available on your PYTHONPATH environment variable? Did you "
            "forget to activate a virtual environment?"
        ) from exc
    execute_from_command_line(sys.argv)

if __name__ == '__main__':
    main()
```

```
./webscraper_project/scrapper/__init__.py
```

```
./webscraper_project/scrapper/__init__.py
```

```
./webscraper_project/scrapper/admin.py
```

```
./webscraper_project/scrapper/admin.py
```

```
from django.contrib import admin
```

```
# Register your models here.
```

```
./webscraper_project/scrapper/apps.py
```

```
./webscraper_project/scrapper/apps.py
```

```
from django.apps import AppConfig
```

```
class ScraperConfig(AppConfig):  
    name = 'scraper'
```

```
./webscraper_project/scrapper/management/__init__.py
```

```
./webscraper_project/scrapper/management/__init__.py
```

```
./webscraper_project/scrapper/management/commands/__init__.py
```

```
./webscraper_project/scrapper/management/commands/__init__.py
```

```
./webscraper_project/scrapper/management/commands/scrapper.py
```

```
./webscraper_project/scrapper/management/commands/scrapper.py
```

```
from django.core.management.base import BaseCommand
from scrapper.services.scrape import scrape_website
from scrapper.models import ScrapedData

class Command(BaseCommand):
    help = "Run the web scraper"
    # Hereda de BaseCommand, lo que permite que este comando sea ejecutable mediante python manage.py <nombre_comando>.

    def handle(self, *args, **kwargs):
        # Ejecuta función
        data = scrape_website()
        print("Scraped Data:", data) # Agrega esta línea para depurar
        # Guarda
        for item in data:
            ScrapedData.objects.create(title=item["title"], url=item["url"])
        # Confirma
        self.stdout.write(self.style.SUCCESS("Scraping completed!"))
```



```
./webscraper_project/scrapper/migrations/0001_initial.py
```

```
./webscraper_project/scrapper/migrations/0001_initial.py
```

```
# Generated by Django 6.0.1 on 2026-01-29 13:47
```

```
from django.db import migrations, models
```

```
class Migration(migrations.Migration):
```

```
    initial = True
```

```
    dependencies = []
```

```
    operations = [
```

```
        migrations.CreateModel(
```

```
            name='ScrapedData',
```

```
            fields=[
```

```
                ('id', models.BigAutoField(auto_created=True, primary_key=True, serialize=False, verbose_name='ID')),
```

```
                ('title', models.CharField(max_length=200)),
```

```
                ('url', models.URLField()),
```

```
                ('created_at', models.DateTimeField(auto_now_add=True)),
```

```
            ],
```

```
        ),
```

```
    ]
```

```
./webscraper_project/scrapper/migrations/__init__.py
```

```
./webscraper_project/scrapper/migrations/__init__.py
```

```
./webscraper_project/scrapper/models.py
```

```
./webscraper_project/scrapper/models.py
```

```
from django.db import models

# Create your models here.
class ScrapedData(models.Model):
    title = models.CharField(max_length=200)
    url = models.URLField()
    created_at = models.DateTimeField(auto_now_add=True)

    def __str__(self):
        return self.title
```

```
./webscraper_project/scrapper/services/__init__.py
```

```
./webscraper_project/scrapper/services/__init__.py
```

```
./webscraper_project/scrapper/services/scrape.py
```

```
./webscraper_project/scrapper/services/scrape.py
```

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

# Para Chrome
from selenium.webdriver.chrome.service import Service
from selenium.webdriver.chrome.options import Options
from webdriver_manager.chrome import ChromeDriverManager

def scrape_website():
    # Configurar Selenium
    options = Options()
    options.add_argument('--headless') # Ejecutar en modo headless
    options.add_argument('--no-sandbox') # Requerido para algunos servidores
    options.add_argument('--disable-dev-shm-usage') # Para evitar errores de memoria

    # ♦ Aquí inicializamos correctamente 'service'
    service = Service(ChromeDriverManager().install())

    # Para Chrome
    # Selenium Manager se encargará de descargar y gestionar el WebDriver
    #service = Service() # No es necesario especificar el ejecutable
    driver = webdriver.Chrome(service=service, options=options)

    # Navegar al sitio web
    url = "https://jorgebenitezlopez.com"
    driver.get(url)
    print(driver.title)
# Esperar a que los elementos estén presentes
try:
    WebDriverWait(driver, 10).until(
        EC.presence_of_all_elements_located((By.CSS_SELECTOR, "h1"))
    )
    titles = driver.find_elements(By.CSS_SELECTOR, "h1")
    urls = driver.find_elements(By.CSS_SELECTOR, "a")
except Exception as e:
    print("Error al encontrar los elementos:", e)
    driver.quit()
    return []

scraped_data = []
for title, link in zip(titles, urls):
    scraped_data.append({
        "title": title.text,
        "url": link.get_attribute("href"),
    })

print("Scraped data:", scraped_data) # Para depuración
driver.quit()
return scraped_data
```

```
./webscraper_project/scrapper/services/scrape_ChatGPT_Corregido.py
```

```
./webscraper_project/scrapper/services/scrape_ChatGPT_Corregido.py
```

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
from selenium.webdriver.chrome.service import Service
from selenium.webdriver.chrome.options import Options
from webdriver_manager.chrome import ChromeDriverManager

def scrape_website(url: str = "https://jorgebenitezlopez.com") -> list[dict]:
    options = Options()
    options.add_argument("--headless")
    options.add_argument("--no-sandbox")
    options.add_argument("--disable-dev-shm-usage")

    service = Service(ChromeDriverManager().install())
    driver = webdriver.Chrome(service=service, options=options)

    try:
        driver.get(url)

        WebDriverWait(driver, 10).until(
            EC.presence_of_all_elements_located((By.CSS_SELECTOR, "h1"))
        )

        scraped_data = []

        # Ejemplo simple: solo h1 (datos coherentes)
        titles = driver.find_elements(By.CSS_SELECTOR, "h1")

        for title in titles:
            scraped_data.append({
                "title": title.text,
                "url": url,
            })

        return scraped_data

    except Exception as e:
        # En Django: logging.exception(e)
        print("Error durante el scraping:", e)
        return []

    finally:
        # ✔ SIEMPRE se ejecuta
        driver.quit()
```

```
./webscraper_project/scrapper/services/scrape_inline.py
```

```
./webscraper_project/scrapper/services/scrape_inline.py
```

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

# Para Chrome
from selenium.webdriver.chrome.service import Service
from selenium.webdriver.chrome.options import Options
from webdriver_manager.chrome import ChromeDriverManager

def scrape_website():
    # Configurar Selenium
    options = Options()

    # OK: headless es correcto para scraping
    options.add_argument('--headless')

    # OK en servidores Linux / contenedores
    # En Windows no es estrictamente necesario, pero no molesta
    options.add_argument('--no-sandbox')

    # OK: evita errores de memoria en entornos limitados
    options.add_argument('--disable-dev-shm-usage')

    # ⚠️ webdriver-manager se ejecuta CADA VEZ que llamas a la función
    # Funciona, pero es costoso.
    # Más adelante convendría inicializarlo una sola vez.
    service = Service(ChromeDriverManager().install())

    # Inicialización correcta del driver
    driver = webdriver.Chrome(service=service, options=options)

    # URL hardcodeada → aceptable para el ejercicio,
    # pero mejor recibirla como parámetro
    url = "https://jorgebenitezlopez.com"
    driver.get(url)

    # ✔️ Útil para depuración, ❌ no ideal en producción
    print(driver.title)

    # ❌ Problema de estilo:
    # Este comentario debería estar indentado como el código
    # (no rompe nada, pero es confuso)
    # Esperar a que los elementos estén presentes
    try:
        # ✔️ Bien: WebDriverWait en lugar de time.sleep
        WebDriverWait(driver, 10).until(
            EC.presence_of_all_elements_located((By.CSS_SELECTOR, "h1"))
        )

        # ⚠️ Aquí hay un problema lógico:
        # Estás obteniendo TODOS los <h1> y TODOS los <a>
        # y luego los unes con zip().
        # No hay garantía de que correspondan entre sí.
        titles = driver.find_elements(By.CSS_SELECTOR, "h1")
        urls = driver.find_elements(By.CSS_SELECTOR, "a")

    except Exception as e:
        # ❌ print en lugar de logging
        print("Error al encontrar los elementos:", e)

        # ❌ driver.quit() aquí...
        driver.quit()
        return []

    scraped_data = []

    # ⚠️ zip(titles, urls) puede:
    # - perder elementos
    # - mezclar datos incorrectos
    for title, link in zip(titles, urls):
        scraped_data.append({
            "title": title.text,
            "url": link.get_attribute("href"),
        })

    # ✔️ útil para depuración
    print("Scraped data:", scraped_data)

    # ❌ driver.quit() solo se ejecuta si NO hubo excepción
    # Si algo falla antes, el navegador puede quedar abierto
    driver.quit()
```

```
return scraped_data
```



```
./webscraper_project/scrapper/tests.py
```

```
./webscraper_project/scrapper/tests.py
```

```
from django.test import TestCase
```

```
# Create your tests here.
```

```
./webscraper_project/scrapper/views.py
```

```
./webscraper_project/scrapper/views.py
```

```
from django.shortcuts import render
```

```
# Create your views here.
```

```
./webscraper_project/webscraper_project/__init__.py
```

```
./webscraper_project/webscraper_project/__init__.py
```

```
./webscraper_project/webscraper_project/asgi.py
```

```
./webscraper_project/webscraper_project/asgi.py
```

```
"""
ASGI config for webscraper_project project.

It exposes the ASGI callable as a module-level variable named ``application``.

For more information on this file, see
https://docs.djangoproject.com/en/6.0/howto/deployment/asgi/
"""

import os

from django.core.asgi import get_asgi_application

os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'webscraper_project.settings')

application = get_asgi_application()
```

./webscraper_project/webscraper_project/settings.py

./webscraper_project/webscraper_project/settings.py

```
"""
Django settings for webscraper_project project.

Generated by 'django-admin startproject' using Django 6.0.1.

For more information on this file, see
https://docs.djangoproject.com/en/6.0/topics/settings/

For the full list of settings and their values, see
https://docs.djangoproject.com/en/6.0/ref/settings/
"""

from pathlib import Path

# Build paths inside the project like this: BASE_DIR / 'subdir'.
BASE_DIR = Path(__file__).resolve().parent.parent

# Quick-start development settings - unsuitable for production
# See https://docs.djangoproject.com/en/6.0/howto/deployment/checklist/

# SECURITY WARNING: keep the secret key used in production secret!
SECRET_KEY = 'django-insecure-v!^=sxx$1xph6#36z(xke2!bs29!$+xx(!0i0*mgfu3jr3unwa'

# SECURITY WARNING: don't run with debug turned on in production!
DEBUG = True

ALLOWED_HOSTS = []

# Application definition

INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'scraper', # Tu nueva app
]

MIDDLEWARE = [
    'django.middleware.security.SecurityMiddleware',
    'django.contrib.sessions.middleware.SessionMiddleware',
    'django.middleware.common.CommonMiddleware',
    'django.middleware.csrf.CsrfViewMiddleware',
    'django.contrib.auth.middleware.AuthenticationMiddleware',
    'django.contrib.messages.middleware.MessageMiddleware',
    'django.middleware.clickjacking.XFrameOptionsMiddleware',
]

ROOT_URLCONF = 'webscraper_project.urls'

TEMPLATES = [
    {
        'BACKEND': 'django.template.backends.django.DjangoTemplates',
        'DIRS': [],
        'APP_DIRS': True,
        'OPTIONS': {
            'context_processors': [
                'django.template.context_processors.request',
                'django.contrib.auth.context_processors.auth',
                'django.contrib.messages.context_processors.messages',
            ],
        },
    },
]

WSGI_APPLICATION = 'webscraper_project.wsgi.application'

# Database
# https://docs.djangoproject.com/en/6.0/ref/settings/#databases

DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.sqlite3',
        'NAME': BASE_DIR / 'db.sqlite3',
    }
}
```

```
# Password validation
# https://docs.djangoproject.com/en/6.0/ref/settings/#auth-password-validators

AUTH_PASSWORD_VALIDATORS = [
    {
        'NAME': 'django.contrib.auth.password_validation.UserAttributeSimilarityValidator',
    },
    {
        'NAME': 'django.contrib.auth.password_validation.MinimumLengthValidator',
    },
    {
        'NAME': 'django.contrib.auth.password_validation.CommonPasswordValidator',
    },
    {
        'NAME': 'django.contrib.auth.password_validation.NumericPasswordValidator',
    },
]

# Internationalization
# https://docs.djangoproject.com/en/6.0/topics/i18n/

LANGUAGE_CODE = 'en-us'

TIME_ZONE = 'UTC'

USE_I18N = True

USE_TZ = True

# Static files (CSS, JavaScript, Images)
# https://docs.djangoproject.com/en/6.0/howto/static-files/

STATIC_URL = 'static/'
```

```
./webscraper_project/webscraper_project/urls.py
```

```
./webscraper_project/webscraper_project/urls.py
```

```
"""
URL configuration for webscraper_project project.

The `urlpatterns` list routes URLs to views. For more information please see:
    https://docs.djangoproject.com/en/6.0/topics/http/urls/
Examples:
Function views
    1. Add an import:  from my_app import views
    2. Add a URL to urlpatterns:  path('', views.home, name='home')
Class-based views
    1. Add an import:  from other_app.views import Home
    2. Add a URL to urlpatterns:  path('', Home.as_view(), name='home')
Including another URLconf
    1. Import the include() function: from django.urls import include, path
    2. Add a URL to urlpatterns:  path('blog/', include('blog.urls'))
"""
from django.contrib import admin
from django.urls import path

urlpatterns = [
    path('admin/', admin.site.urls),
]
```

```
./webscraper_project/webscraper_project/wsgi.py
```

```
./webscraper_project/webscraper_project/wsgi.py
```

```
"""
WSGI config for webscraper_project project.

It exposes the WSGI callable as a module-level variable named ``application``.

For more information on this file, see
https://docs.djangoproject.com/en/6.0/howto/deployment/wsgi/
"""

import os

from django.core.wsgi import get_wsgi_application

os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'webscraper_project.settings')

application = get_wsgi_application()
```